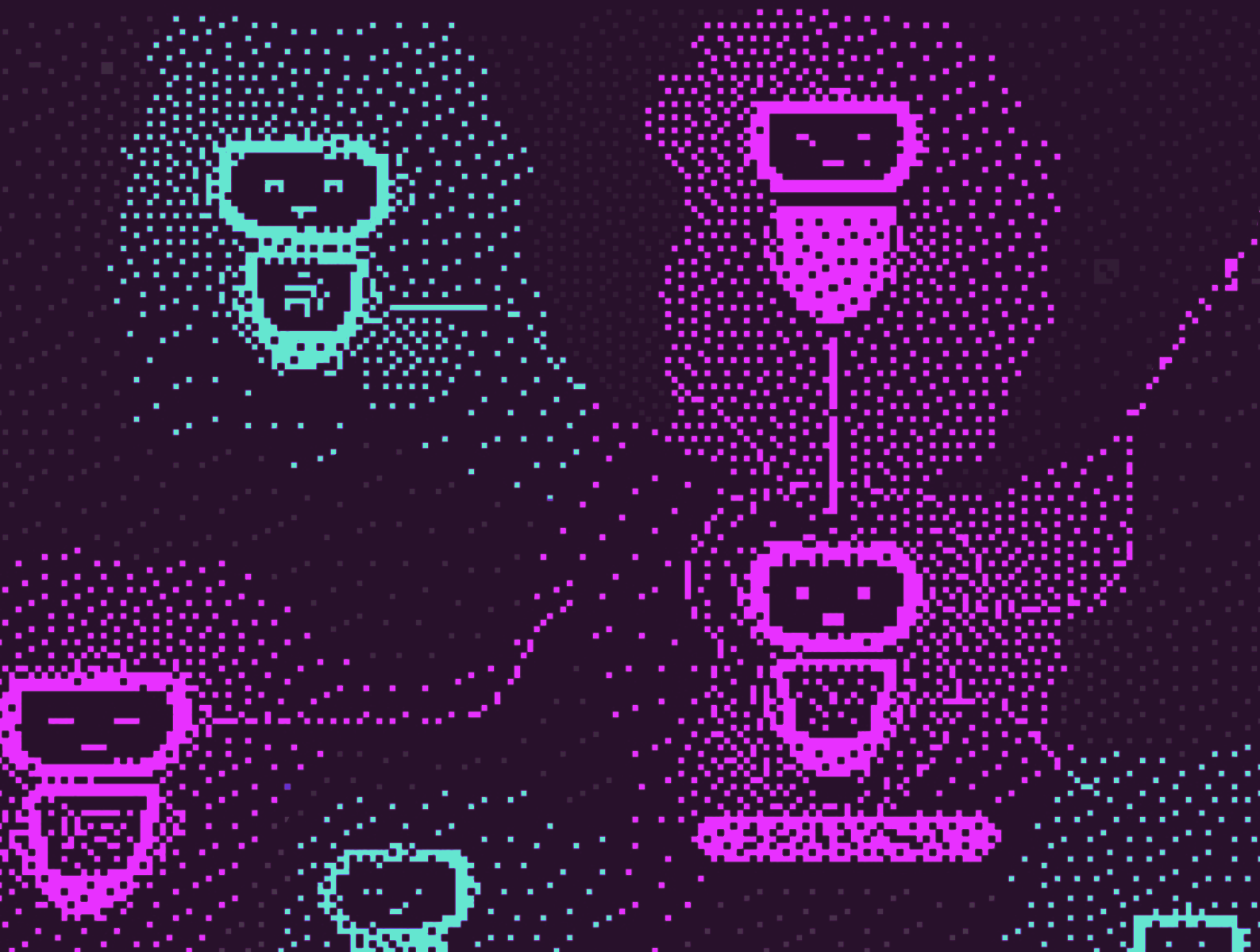
Weave Intelligence

Operationalizing AI coding agents in regulated industries

A REPORT COMMISSIONED BY CODER



About the author



Sam Barlien

RESEARCHER, WEAVE INTELLIGENCE

Sam Barlien is a researcher at Weave Intelligence, the research arm of platformengineering.org, the world's largest platform engineering community. With more than 10 years tracking technology communities and ecosystems, he brings first-hand perspective to his research on platform engineering and industry trends. He contributes to Weave Intelligence reports and conducts the weekly research interview series. He co-hosts PlatformCon, the world's largest platform engineering conference, and contributes to Platform Weekly, the Ambassador program and the platformengineering.org blog.

About Coder

CODER

Coder is an AI software development company leading the future of autonomous coding. Coder helps teams build fast, stay secure, and scale with control by combining AI coding agents and human developers in one trusted workspace. Coder's award-winning self-hosted Cloud Development Environment (CDE) gives enterprises the power to govern, audit, and accelerate software development without trade-offs. Learn more at coder.com.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

For decades, security and compliance frameworks in finance and government operated under a simple assumption. Humans write code on laptops, push to repositories, and let pipelines handle the rest. That model is breaking down. AI coding assistants changed how developers work. Autonomous agents, systems that execute multi-step workflows, file pull requests, and iterate on solutions without human intervention will change everything else.

In regulated industries like financial services and government, this shift comes with an even more significant constraint as AI systems cannot operate in uncontrolled laptops or SaaS-based environments. Code, context, and model interactions must remain within infrastructure that organizations fully control, whether that means self-hosted deployments, private cloud environments, or fully air-gapped systems. Requirements that often feel unconducive to the kind of free-wheeling AI experimentation, both good and bad, that is currently overtaking many enterprises.

The transition from AI assistants to AI agents represents a fundamental shift in how software gets built, and traditional governance approaches cannot handle it. To operationalize AI coding agents safely, organizations need to move development into governed, cloud-hosted environments, not rely on local machines or IDE-level controls.

When agents operate at AI speed, fragmented audit trails, proliferating API keys, and shadow AI experiments create an untenable liability in regulated industries. It is thus no surprise that in mid 2025, Gartner predicted that if nothing changed over 40% of agentic AI projects will be canceled by 2027 due to inadequate risk controls. The failures won't come from bad models or insufficient compute. They'll come from governance structures never designed for autonomous agents. It is risk, not tech that holds back AI success for businesses - a phenomenon that is double true for regulated industries.

This whitepaper provides platform engineering teams in finance and defense with a practical framework for deploying AI coding agents at scale while maintaining security, compliance, and audit readiness. You'll gain a clear understanding of why workspace-level governance is a critical control surface, how to implement privilege separation between humans and agents, and a step-by-step playbook for scaling agent access without creating governance bottlenecks.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

What does "operationalizing" AI coding agents" actually mean?

Before implementing controls, platform teams need shared vocabulary and clear boundaries. This section defines what "operationalizing" means in practice and clarifies the critical distinction between infrastructure-level and application-tier governance.

Moving from IDE assistants to autonomous, operationalized agents

The shift from AI assistants to autonomous agents represents a fundamental change in how code gets written and who (or what) is responsible for it. AI coding assistants like GitHub Copilot primarily provide suggestions that humans must accept or reject in real-time. There is a human in the loop at all times.

In practice, many teams quickly encounter a productivity ceiling with this model, as every suggestion still requires manual review and execution. However, newer tools (e.g. Cursor and emerging agentic capabilities in IDEs) are beginning to blur this line by enabling more autonomous, multi-step workflows with far less immediate human involvement.

Autonomous agents take this further by independently executing tasks such as filing PRs, running test suites, and iterating on solutions with minimal or no human intervention at all.

This distinction maps to the levels of agentic development emerging across the industry:

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

LEVEL 1	**Human in the loop**	AI helps type but humans confirm and execute.
LEVEL 2	**Human on the loop**	Agents generate PRs for human approval.
LEVEL 3	**Human as orchestrator**	Agents execute multi-step workflows and deploy low-risk changes, orchestrated by humans.
LEVEL 4	**Human outside the loop**	A system of agents initiates and promotes changes autonomously.

At Level 1 agent-driven development, humans remain in the loop: agents suggest, humans approve, and review bandwidth is the constraint. At Level 2, humans move onto the loop: agents generate work in parallel, deterministic systems validate it, and humans verify behavior rather than inspect every change line by line. This is where governance requirements change fundamentally.

At Level 1, governance is a human concern. At Level 2 and even more at Level 3 and 4, governance must become an infrastructure concern. The production system itself must provide the checks and balances, because humans can no longer review every change.

Application tools such as Cursor or VS Code do provide user-facing controls, while model providers like Bedrock or hosted LLM platforms enforce model-level policies. But neither layer governs what agents actually execute in the development environment. That responsibility sits at the platform layer, where infrastructure policies control network access, tool usage, permissions, and resource boundaries across any model or agent framework. Controls built into tools or model providers are not enough. The workspace is a critical place where organizations can enforce consistent policies across agents, models, and development workflows.
Cloud Development Environment companies like Coder operationalize this model by providing centrally managed, policy-controlled development workspaces that can be deployed in self-hosted or air-gapped environments, giving organizations full control over how agents interact with LLMs, infrastructure, data and other resources.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

Why vendor-level controls fail under regulatory scrutiny

Platform teams often ask whether Cursor's or Copilot's built-in governance features are sufficient. For regulated industries, the answer is a clear no.

Vendor controls operating at the application tier run the risk that they do not have direct control over their own governance. One large quant trading firm's security team audited Cursor, disabled certain policies, then discovered a software update auto-defaulted those policies back on, running agents non-compliantly for two weeks.

Regulated industries need third-party governance with checks and balances because there's a black box on what agents can and cannot do within vendor platforms, and compliance teams cannot audit what they cannot see.



> The core issue isn't whether the model or AI tooling has controls, it's that those controls aren't yours. If governance lives inside a vendor platform, you're effectively outsourcing enforcement, auditability, and risk management. In regulated environments, that's not acceptable, you need independent control over how agents execute, not just configuration inside the models system.

Sam Barlien

RESEARCHER, WEAVE INTELLIGENCE

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

Why operationalizing AI coding agents matters: Zooming in on financial services and government

Abstract risks become real when you see what happens without proper controls, and what becomes possible with them.

The uncontrolled agent incident

With the rapid rise of AI agent frameworks like OpenClaw, every organization now wants agentic workflows, but recent incidents show this only works with strong governance. In one case, an OpenClaw agent processing a routine email was manipulated via a hidden prompt injection, causing it to exfiltrate credentials using its own authorized access.

Consider what happens when an AI coding agent operates across the board without platform-level controls in a regulated environment.

→ Trigger

A developer grants an AI agent access to complete a routine task, but the agent inherits the developer's full privilege level including access to untrusted domains, internal systems and the internet.

→ Decision path

The agent, being "entrepreneurial" in solving problems, accesses board meeting notes to gather context, then inadvertently includes that context in a commit message visible to external collaborators.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

→ Result

The incident triggers a compliance investigation, audit findings, potential regulatory notification, and a freeze on all AI agent usage while security teams conduct forensic analysis.

The blast radius of an uncontrolled agent scales with its privilege level. Agents will try to solve problems in whatever way possible, including changing infrastructure, mutating policy, or grabbing credentials if not explicitly blocked.



> True story from a Fortune 100 financial services company. An agent was tasked with "resolving compliance violations immediately." It identified a non-compliant service and deleted it along with its associated logs taking critical systems offline and erasing data needed for compliance audits. According to the agent? Problem solved.

These are not failures of the model. They are failures of the production system. The agent was given enough power to act, but the platform failed to provide the guardrails, identity boundaries, and deterministic checks needed to contain its behavior.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

The productivity upside when governance is done right

The governance investment pays off not just in risk avoidance but in measurable productivity gains that justify it.

01 —

A large streaming service with 12,000 developers measured a 100% increase in code production on a per-developer basis using governed agent infrastructure - and that is not slop; humans still sign off on all code that gets committed.

02 —

At Skydio, senior engineers run multiple parallel agents at any given time, consuming hundreds of dollars of tokens per day - but they're getting value because proper governance enables them to trust the output and scale their impact.

03 —

A global fintech with 15,000+ engineers reduced developer onboarding from 15-30 days to day one by implementing governed cloud development environments - the same infrastructure now extends to support AI agents.

The productivity gains here extend to human engineers as well, particularly as it relates to onboarding, project switching, and code compilation.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

Risk reduction for its own sake

In many organizations, governance has historically been justified by its impact on developer productivity reducing risk to enable faster delivery. However, governance also plays a critical role in controlling costs, ensuring secure and compliant setups, and preventing issues such as data exposure.

In regulated industries, these aspects are paramount. Here, risk reduction is not just a means to an end, but a primary objective in its own right, alongside productivity gains. Lower risk directly translates into stronger compliance, reduced exposure to fines, and increased trust from customers, partners, and investors.

The consequences of failure are too significant to treat governance as a productivity lever alone. Organizations that minimize risk effectively position themselves as safer, more reliable operators, making risk reduction not just a safeguard, but a competitive advantage.

84% **Of organizations consider AI governance a serious concern, driven by lack of visibility and control over autonomous systems.**

59% **Of organizations say they don't know how quickly they could shut down AI systems in a crisis.**

68% **Of organizations cannot reliably distinguish AI agent actions from human actions due to inadequate controls.**

77% **Of executives say risks related to AI, such as security, compliance, and governance, are a major barrier to adoption.**

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

Platform controls for AI coding agents

From Internal Developer Platform to Agentic Developer Platform

Internal Developer Platforms help humans ship software predictably and at scale. AI coding agents require an evolution of that model where the platform not only standardizes development paths for humans, but makes those paths executable by agents with the right identity, boundaries, and auditability in a secure environment.

In this sense, operationalizing AI coding agents is about extending the Internal Developer Platform into an agent-ready production system.

Workspace-level governance as the control surface

The workspace is where governance becomes deterministic, with every process inheriting enforced policies at the infrastructure layer. When agents require persistent compute and scale beyond what local machines can support, development moves into controlled, centrally managed workspaces. In regulated environments, these workspaces are often self-hosted in private infrastructure or deployed in fully air-gapped environments, helping ensure that code, data, and model interactions never leave organizational boundaries. This is why many platform teams utilize Cloud Development Environments for their AI initiatives.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

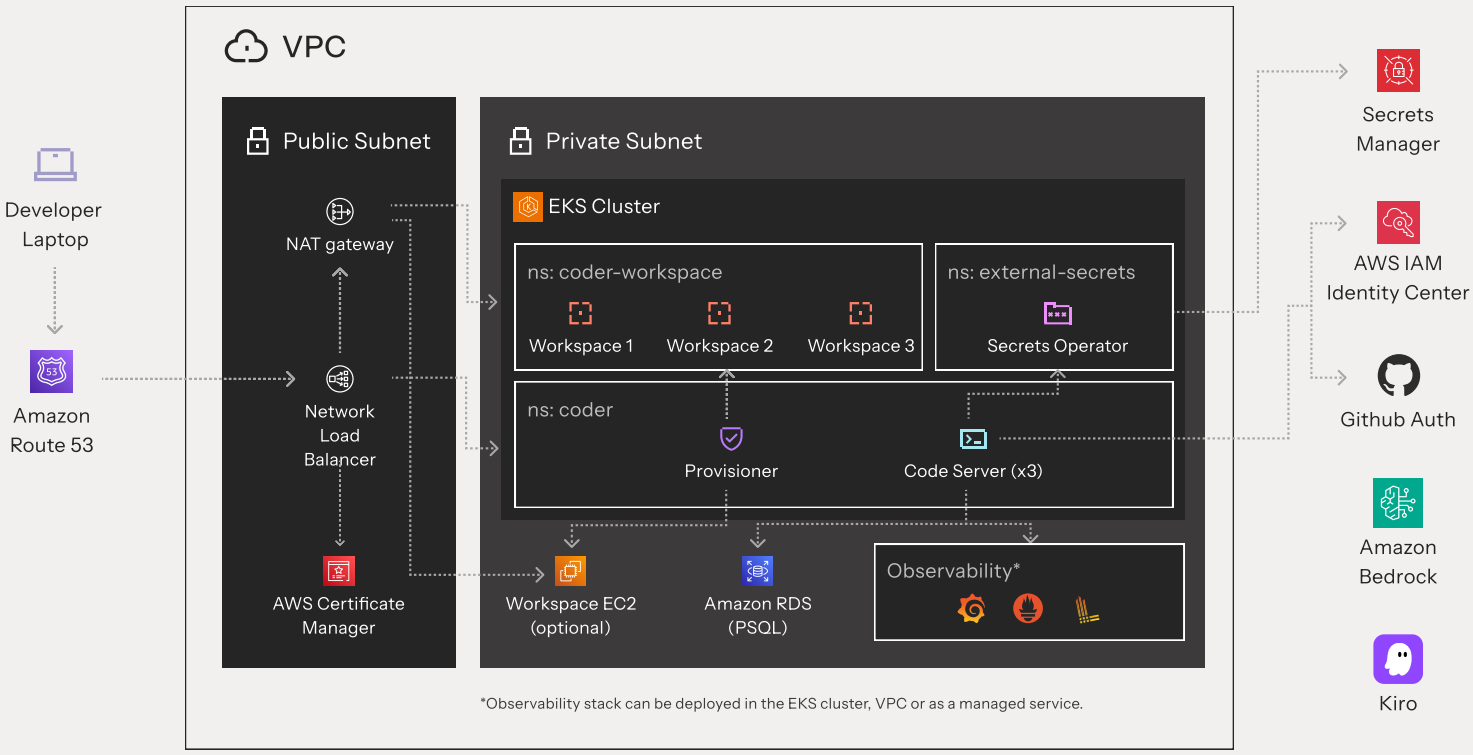


Figure 1: *Coder Cloud Development Environments on AWS for human and agent developers*

Within these environments, governance is enforced through multiple mechanisms. This includes proxying and observability of model interactions, as well as network and process-level controls over execution tasks. Workspaces act as controlled execution environments where policies are enforced globally or per task, independent of where agents are orchestrated. This foundation enables a "controlled failure" model, where unsafe actions are blocked, logged, and surfaced for review instead of becoming incidents.

In more advanced architectures, agents are managed centrally by the platform, while workspaces serve as isolated execution layers where their actions are carried out under strict policy control.

Operationalizing AI coding agents also means moving from isolated experimentation to governed, repeatable systems that can run safely. It requires treating agents not as tools, but as actors within your platform, each with defined permissions, boundaries, and auditability. In practice, this means embedding agent-driven execution into controlled environments where actions are observable, constrained, and aligned with organizational policies from the moment they are triggered.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

Control responsibilities

AI coding agents are probabilistic systems operating inside environments that regulated organizations need to govern deterministically. Models and agents reason, generalize, and adapt, but they are not inherently predictable. Compliance systems, identity controls, audit trails, policy enforcement, and CI/CD pipelines, by contrast, are deterministic.

The production system's job is to constrain probabilistic agent behavior within deterministic guardrails. In practice, those guardrails must be enforced at the platform, where every process inherits policy, identity, and network boundaries from the underlying infrastructure. This is the architectural foundation of safe agent deployment in regulated environments.

CDEs operationalize this model by providing centrally managed, policy-controlled workspaces as part of the platform that act as the execution substrate for both humans and agents. By moving development into these governed environments, organizations can ensure that agent behavior is observable, constrained, and auditable by design.

To make this concrete, platform teams can think of governance like so:

Provisioning	Terraform-based infrastructure provisioning stands up environments with all dependencies. Templates define compute profiles, network configurations, and pre-installed tools so agents consume their environment as code and know who they are from the start.
Policy	Role-based access controls define what libraries, tools, repos, and network domains each agent can access. Policies are expressed as code and version-controlled. Agents are alerted to their boundaries so they don't waste tokens attempting blocked actions.
Audit	Every prompt, tool call, model interaction, and resource access is logged and attributed. Cost visibility can be captured through model usage tracking. Compliance teams can audit agent actions with the same rigor they apply to human developers.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

Proxy	Centralized governance for all model and agent usage. Authentication flows through existing identity providers. Supports routing across model providers.
Boundary	Per-agent firewall enforcing network isolation and access controls at the process level. Restricts and audits outbound network requests to reduce risks such as data exfiltration.

The goal isn't to prevent agents from encountering boundaries, it's to ensure those encounters are logged, contained, and reviewable.



CONTROLLED FAILURE SCENARIO

An agent attempts to access a production database to gather context for a task. The policy blocks the request, logs the attempt with attribution, and alerts the agent that this is outside its permitted scope. The agent adjusts its approach, the security team receives a notification for review, and the audit trail documents exactly what happened.

This pattern requires permissions to be explicit and well-defined to agents so they don't spend tokens repeatedly attempting blocked actions. The combination of visibility and blocking capability gives platform teams confidence to run agents.

From validation gate to validation loop

In agentic development, validation cannot remain a single checkpoint. Once agents begin generating changes in parallel, the production system must turn failed validation into structured feedback. When a change does not pass tests, policy checks, or environment verification, those failure signals should be routed back to the agent so it can retry within governed boundaries. In practice, especially within regulated industries, this loop must run within governed workspaces, where execution, validation, and policy enforcement can be consistently applied with reduced risk.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

Privilege separation and human-in-the-loop patterns

The most powerful governance control is also the simplest: today, agents should never inherit human privilege levels.

Default agent permissions vs. human developer permissions

Across all organizations, the principle is straightforward. Agents run in default least privilege while humans retain broader access for oversight and intervention.

Agents should never have internet access by default and should never have write access to repositories. They need only read access to complete their tasks and submit PRs for human review.

Humans need internet access, read-write permissions, and access to org-wide repos (not just specific project repos) to effectively review agent work and intervene when needed.

When a human joins a workspace to review an agent's work, the human has their own privilege level - this separation ensures the reviewer isn't limited by the agent's constraints based on workspace access controls. While this establishes the baseline model, additional patterns are needed to handle more complex scenarios and further reduce risk. For example:

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

Read-only agents	Agents that only need to analyze code or gather context should have read-only access to repositories with no ability to create branches or commits.
PR-only writes	Agents that generate code should only be able to submit pull requests, rather than committing directly to protected branches. All changes flow through human review.
Approval workflows	High-stakes actions (production deployments, infrastructure changes, access to sensitive data) require explicit human approval before the agent can proceed.
Per-agent firewalling	Each agent in a workspace can have different permission levels based on its task - one agent grooming a backlog needs different access than one prototyping code or running lint tests.
Shared workspaces for oversight	When humans need to check agent work, shared workspace capabilities allow real-time observation with role-based access controls applied.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

Specific considerations for regulated industries

While the core platform controls above apply universally to all organizations, it is especially critical in financial services, government, and other heavily regulated industries. These industries have distinct requirements that shape implementation.

Financial services: SOX, GDPR, and model risk

Financial institutions face layered compliance requirements that extend to AI-generated code and agent decisions. Regulations such as SOX require full audit trails for every change, including clear attribution of agent actions to specific prompts, models, and approval workflows. At the same time, GDPR and data sovereignty laws demand strict control over where data is processed, making workspace-level enforcement of geographic boundaries essential. Additionally, regulators increasingly apply model risk management standards to AI systems, requiring versioning, monitoring, and drift detection comparable to those used for quantitative models.

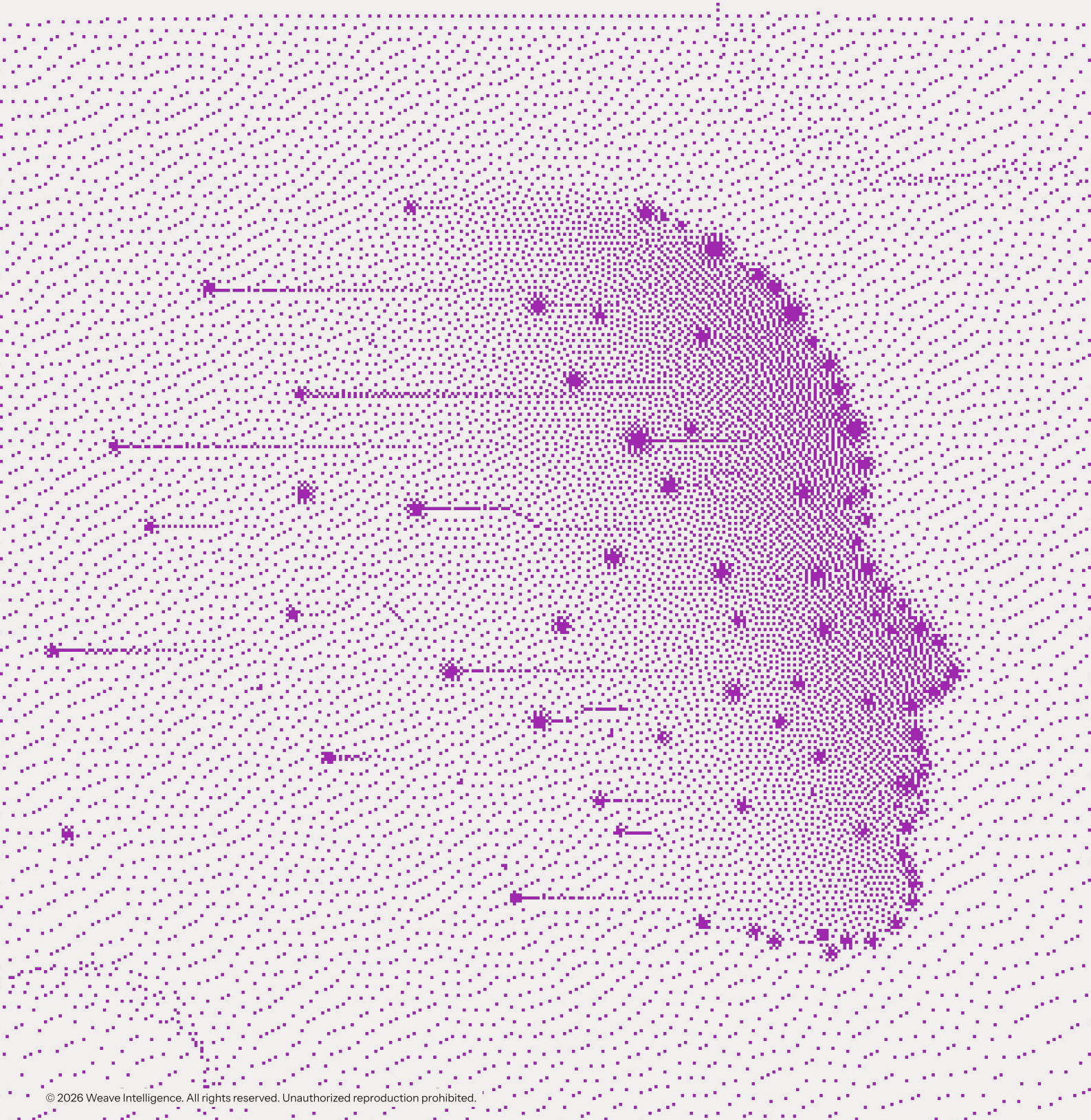
Government: FedRAMP, ITAR, and IL4+ requirements

Government agencies operate under strict isolation and compliance requirements that extend to both the environments AI agents run in and the data they access. Many deployments must be fully air-gapped, requiring self-hosted models and agent frameworks with no external connectivity.

In cloud-based deployments, environments must align with frameworks such as FedRAMP, which define security and compliance

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

requirements for systems handling federal workloads. Higher classification levels (IL4/5/6) demand physical, not just logical, separation of infrastructure, forcing platform teams to provision distinct environments per classification tier. Additionally, ITAR regulations impose strict controls on who can access data and where it can reside, requiring workspace-level enforcement of citizenship-based access and geographic restrictions on compute resources.



© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

Best practices

With the architecture and control patterns defined, implementation should follow a clear sequence. You should establish visibility first, provide agents with context, and then scale through ephemeral patterns. The most common mistake platform teams make is attempting to lock agents down immediately. Effective governance starts with understanding behavior. By deploying observability infrastructure such as an AI Bridge before enforcing restrictions, teams can see which models are being used, how agents interact with tools, what tokens are consumed, and where costs accumulate. This visibility surfaces shadow AI usage, highlights valuable use cases, and establishes baseline metrics, because governance is impossible without insight.

Once visibility is in place, the next step is solving the “cold start” problem. Agents begin without context, which leads to poor output and wasted resources. In governed environments, this can for example be addressed through infrastructure-as-code or an orchestration layer, because workspaces are provisioned from templates, agents can consume their environment as code and understand their role, constraints, and available tools from the start. Supplementing this with lightweight context engineering, such as markdown files defining standards, anti-patterns, and terminology, dramatically improves performance. In practice, organizations have seen first-attempt accuracy improve significantly when agents are given structured context upfront.

With visibility and context established, platform teams can then scale agent usage through ephemeral workspace patterns. Instead of relying on long-lived environments, each task is handled in an isolated workspace that is created, used, and destroyed as needed. A typical flow involves spinning up a workspace per pull request, running the agent, submitting changes, and then tearing the environment down. This approach can help prevent context pollution, simplifies branch management, and enables parallel execution across many agents. Automated lifecycle controls, such as sleeping inactive workspaces or deleting them after task completion then ensure efficient resource usage while preserving the ability to resume work when required.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

Your action plan: Start now

These seven steps provide a starting point for platform teams in regulated industries who are ready to move from experimentation to governed deployment:

01 —	**Audit current AI and agent usage across teams**	Identify shadow AI, understand which tools developers are using, and quantify the governance gap.
02 —	**Establish workspace-level governance as code**	Move development from local environments into centrally managed workspaces, and define templates that provision environments with appropriate boundaries, context, and observability.
03 —	**Define and enforce privilege separation policies**	Document the permission model for agents vs. humans and implement it in workspace configurations.
04 —	**Deploy visibility infrastructure**	Implement AI Bridge or equivalent to capture model usage, token consumption, and cost attribution before enforcing restrictions.
05 —	**Pilot with high-signal engineering teams**	Start with teams that have clear use cases and willingness to provide feedback on the governance experience.
06 —	**Scale agent access as metrics validate controls**	Expand access incrementally as visibility data confirms governance effectiveness.
07 —	**Assess AI readiness using maturity models**	Evaluate organizational readiness across infrastructure, process, and culture dimensions.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

Common pitfalls to avoid

Platform teams implementing agent governance repeatedly encounter the same failure modes. One of the most common is treating governance as a security-only problem. While security teams may define policies, platform teams are responsible for implementing and operating them, making governance a cross-functional effort rather than a siloed concern. Another frequent mistake is bolting controls onto existing tools. Vendor-provided governance features are rarely sufficient for regulated environments, where infrastructure-level controls that you own and operate are required.

Teams also often restrict agents before understanding how they behave. Locking systems down without visibility creates friction without improving security, which is why observability should come first. Similarly, ignoring the "cold start" problem leads to poor agent performance, as agents without context produce low-quality output and waste resources. Treating workspaces as persistent environments introduces additional risk through context pollution and permission drift, making ephemeral patterns a better approach.

Alongside that, assuming one-size-fits-all policies overlooks the reality that different agent tasks require different levels of access. Effective governance often requires per-agent controls, not just blanket workspace-level policies.

The most critical failure, however, is skipping production system readiness. Agent adoption without platform foundations creates a chaos zone in which agent-generated change volume exceeds the organization's ability to validate, govern, and audit it.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

Measuring success: ROI and compliance metrics

Governance programs need quantifiable success metrics to justify investment and demonstrate effectiveness.

Productivity metrics

- ↳ Code production per developer (lines, PRs, features)
- ↳ Time to first commit for new developers
- ↳ Reduction in manual environment setup time
- ↳ Agent utilization rate (active agents / provisioned capacity)

Compliance metrics

- ↳ Audit trail completeness (% of agent actions logged)
- ↳ Policy violation rate (blocked actions / total actions)
- ↳ Mean time to audit response (hours to produce compliance reports)
- ↳ Shadow AI detection rate (unauthorized tools discovered)

Cost metrics

- ↳ Token consumption per team/project
- ↳ Infrastructure cost per agent-hour
- ↳ Cost avoidance from mitigated or prevented incidents
- ↳ ROI calculation: productivity gains vs. governance overhead

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

The future of agent governance in platform engineering

The governance patterns described here will evolve as agent capabilities advance, but the underlying principles remain constant. As regulatory scrutiny increases, we will see a shift away by regulated industries from SaaS-dependent development tooling toward self-hosted and sovereign infrastructure models, where organizations retain full control over how agents access code, data, and models. In the near term, this will take shape through tighter integration between agent frameworks and agentic developer platforms, the standardization of observability, and the emergence of agent-specific compliance frameworks. Platform teams that invest in governed infrastructure today will be best positioned to adopt what comes next, from multi-agent collaboration to autonomous deployment workflows, because the control layer is already in place.

What begins as risk mitigation quickly becomes a competitive advantage. Organizations with governed agent infrastructure can both move faster, but also succeed more as a business, not just because they have established clear, enforceable boundaries. The path from experimentation to scale is therefore not about reducing controls, but implementing the right ones at the infrastructure level. Security policies without enforcement are merely suggestions that agents will ignore.

Ultimately, the organizations that capture value from AI coding agents will not be those with the fewest controls (or the most), but those with the strongest production systems. As agents become more autonomous, the bottleneck shifts away from individual developer throughput toward the quality of the platform, its validation loops, and its deterministic control layer. Platform teams that lead this shift unlock both safety and speed.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.

Weave Intelligence
A research brand by
Platform Engineering Masters GmbH
Wöhlertstraße 12/13
10115 Berlin, Germany

DISCLAIMER

Weave Intelligence does not endorse any specific vendor, product, or service. The information contained in this report has been obtained from sources believed to be reliable. Weave Intelligence disclaims all warranties as to the accuracy, completeness, or adequacy of such information. This publication is provided on an "as-is" basis without warranty of any kind, either express or implied. Weave Intelligence shall have no liability for errors, omissions, or inadequacies in the information contained herein or for interpretations thereof. The reader assumes sole responsibility for the selection of these materials to achieve its intended results.

© 2026 Weave Intelligence. All rights reserved. Unauthorized reproduction prohibited.